

WorkMonitor Project Concepts

Documentation

This document explains the main concepts, technologies, and modules used in this HR Management project. It is designed to be clear, professional, and suitable for PDF export.

1. Project Overview

This project is an employee management and HR operations system built with Next.js. It helps manage employees, attendance, tasks, leave requests, expenses, notifications, payroll, and reports. The application is designed for three roles: admin, HR, and employee.

2. Main Technologies Used

- Next.js 16: full-stack React framework used for routing, server components, API routes, and app structure.
- React 19: component-based user interface library.
- TypeScript: adds static typing and safer code.
- Tailwind CSS: utility-first styling system.
- shadcn/ui and Radix UI: reusable accessible UI components.
- MongoDB + Mongoose: database and schema-based modeling.
- Cloudinary: image upload and media hosting.
- PDFKit: used for generating salary slip PDFs.
- Zod: validation library.
- React Hook Form: forms and validation.
- Lucide React: icons.

3. Project Structure

- src/app: app routes, pages, API endpoints, login flow, and server actions.
- src/components: reusable UI components like headers, tables, and layout.
- src/lib: shared logic and helper modules such as authentication, database connection, payroll calculations, store, and config.
- src/models: Mongoose schema definitions.
- src/hooks: reusable hooks like data table and search logic.
- src/styles.css: global theme styling.

4. App Router and Page Flow

The app uses the Next.js App Router. The main routes include:

- /login: sign-in page
- /dashboard: overview screen
- /employees: employee listing and management
- /attendance: attendance tracking
- /tasks: employee task assignment
- /leaves: leave request and approval flow
- /expenses: expense requests and review
- /salary-slips: payroll and payslip generation
- /holidays: holiday calendar
- /reports: summaries and analytical views
- /notifications: system alerts
- /profile: employee information and image upload
- /settings: configurations

5. Authentication and Authorization

Authentication is handled by `loginAction()` in the server action layer. The user enters a username or email and password. The backend checks admin credentials from environment variables, HR credentials from environment variables, and employee credentials stored in the `Employee` collection.

After successful login, the user data is stored in `localStorage` under the key `ems_auth_v1`. The application reads the current user using `getCurrentUser()`.

Role-based authorization uses the `Role` type: `admin`, `hr`, `employee`. The `ROLE_MENUS` object defines which pages each role can access. This is the core concept behind access control in the project.

6. State Management

This project does not use `Redux`. Instead, it uses a lightweight custom store pattern in the `lib/store.ts` file.

- `localStorage` session storage for authentication state.
- `useSyncExternalStore` for reactive database-like state updates.
- custom listener system to re-render components when data changes.
- `useDB()` hook to fetch and monitor the employee database state.
- `useGlobalSearch()` for global search state.

This is a simple but effective pattern for small-to-medium internal business applications.

7. Database and Mongoose Models

MongoDB is the database used. Mongoose schemas model each object. The database connection file connects to MongoDB using MONGO_URI from the environment.

- Employee: employee personal information and employment details.
- Task: task assignment, status, progress, reviews, comments.
- Attendance: login/logout records, working time, productivity.
- Leave: leave types, dates, review status, comments.
- Expense: reimbursement requests with approval status.
- Notification: alerts sent to users.
- Holiday: company holiday definitions.
- SalarySlip: payroll data and payslip details.
- Setting: system configuration like maintenance mode.

8. Server Actions and API Routes

The project combines Server Actions and route handlers.

- login
- employee creation and management
- task creation and review
- leave approval and rejection
- expense review and reimbursement
- notification sending and reading
- holiday maintenance
- payroll generation

API routes under `src/app/api/` are used for endpoints like attendance, reports, salary slips, and support functions. The app uses Next.js API route handling to expose backend logic through URLs.

9. Data Table and Search Pattern

The hook `useDataTable.ts` is a reusable pattern for managing search, sorting, and pagination.

- search filtering by text fields
- sort toggle by field name
- pagination with page size and total pages
- start and end index calculation
- filtering and sorted result generation

This hook is used across dashboard, employees, tasks, leave, attendance, reports, holidays, and others.

10. Employee Management

The employee module stores details like name, email, department, designation, joining date, mobile, salary, status, and avatar.

- adding new employees
- updating data
- deleting employees
- viewing employee details
- employee-specific dashboards

11. Attendance System

Attendance is tracked by session records that capture login and logout events. The app stores attendance with fields such as date, loginTime, logoutTime, workingHours, status, productivity, and session histories.

- login and logout tracking
- total working duration calculation
- presence or absence status
- productivity score
- attendance history for each employee

12. Task Management

Tasks are assigned to employees with priority, status, date ranges, comments, and review outcome.

Task lifecycle:

- assigned
- in progress
- completed
- reviewed

HR can review and rate the performance of submitted tasks.

13. Leave Management

Leave requests include type, reason, start date, end date, total days, and review workflow.

- pending
- hr_approved
- hr_rejected
- admin_approved
- admin_rejected
- cancelled

This models the leave approval hierarchy where HR and admin review requests before final approval.

14. Expense Management

Employee expenses are stored and reviewed before reimbursement. Fields include category, amount, date, receipt URL, description, and status.

Typical categories include travel, office supplies, client meetings, food, equipment, and other.

15. Holiday Management

The system can define public and special holidays. The Holiday schema contains name, type, start date, end date, and active status.

- company holiday
- optional holiday
- restricted holiday
- custom holiday

16. Notifications and Alerts

Notifications are central to the system. They are sent for tasks, approvals, reimbursements, broadcasts, and reminders.

- recipient-based notifications
- read/unread state
- system broadcasts to many users
- notification module and action URL links

17. Payroll and Salary Slip Concepts

Payroll logic is defined in the payroll library. Important concepts include:

- annual CTC and monthly CTC
- basic salary and HRA
- conveyance, medical allowance, special allowance
- PF and ESI contribution
- TDS and professional tax
- LWP deduction
- gross salary and net salary
- amount in words conversion

The project computes earnings and deductions and generates salary slip data used in payslip creation.

18. PDF and Salary Slip Generation

The application includes PDF support for salary slips using PDFKit. Salary slip endpoints exist under the API folder. This means the system can generate a payslip document for employee viewing or downloading.

19. Cloudinary Image Upload

The profile page supports uploading a profile image. The server action uploads the image to Cloudinary. This is used for employee profile images, admin, HR, and employee avatars, and visual identity in the system.

20. Maintenance Mode

There is a maintenance mode concept used in the app layout. If maintenance mode is active, non-admin users are redirected to a maintenance page. This is useful for system updates and downtime handling.

21. UI Design and Layout Concept

The interface uses a modern administrative dashboard design with:

- sidebar navigation
- top header with profile and actions
- card-based dashboard metrics
- tables with sorting and pagination
- forms with validation
- alerts and toasts
- role-based layout changes

22. Real-World Business Purpose

The project reflects a complete HR operations workflow. Its main business goals are:

- track employees and their status
- manage attendance and productivity
- assign and monitor tasks
- handle leave approvals
- process employee expense claims
- maintain company holidays
- generate payroll and salary slips
- notify employees about updates
- provide reports for management decisions

23. Summary

The project combines modern frontend architecture, database modeling, role-based access control, and HR workflow automation. The most important concepts are: Next.js app architecture, role-based access control, MongoDB with Mongoose, server actions and API routes, UI state and search logic, payroll calculation and payslip generation, notifications, and employee lifecycle.

24. How to Use This Document

This document is suitable for PDF export. You can open it in a browser and print it to PDF, or open it in Word and save as PDF.

End of Document